IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2026, No. 4, pp. 632–655. DOI:10.46586/tches.v2026.i4.632-655

TAME: A Deep Learning-based Blind Side-Channel Analysis Framework

Jintong Yu, Yuxuan Wang, Pei Cao*, Zixin He, Yubo Zhao, Zhiliang An, Yipeng Shi, Chi Zhang and Dawu Gu*

School of Computer Science, Shanghai Jiao Tong University, Shanghai, China,
jintongyu@sjtu.edu.cn

Abstract. Side-Channel Analysis (SCA) methods critically depend on access to known plaintext/ciphertext, an assumption violated in many practical scenarios such as OCB encryption mode and the EMV payment protocol. Blind SCA addresses this challenge by operating without plaintext/ciphertext, relying instead on a three-phase pipeline whose effectiveness hinges on the quality of trace labeling methods. Existing deep learning-based Blind SCA (DL-BSCA) method achieves promising performance but still suffers from two fundamental limitations: (1) multivariate clustering induces an excessively large joint Hamming weight space with exponential growth which degrades DL-BSCA's performance due to the curse of dimensionality; and (2) all labeled traces are treated with uniform weight during training, ignoring the class information implied in the geometric space, such as the difference in labeling accuracy between samples close to cluster centers and those near cluster boundaries.

To address both limitations, we propose a new DL-BSCA framework comprising two mutually reinforcing components. First, we introduce Multi-point Cluster Deep R epresentation (MCDR) labeling, which performs clustering in a compact learned feature space rather than the raw Hamming weight space, mitigating noise sensitivity and dimensionality explosion. Second, we propose Trust Anchor Meta-weight Estimation (TAME), a bilevel optimization framework that constructs a high-confidence validation set from cluster-interior traces, termed Trust Anchor (TA) set, and automatically learns per-sample weights guided by validation loss, effectively suppressing the influence of mislabeled traces. Better cluster separation from MCDR yields higher-quality TA set, which in turn strengthens TAME's meta-weight supervision. Experiments target three standard cryptographic algorithms, namely AES, ASCON, and Kyber, which demonstrate that our approach recovers the secret key using at most 8000 traces, while prior methods fail with guessing entropy consistently at least 3. Under desynchronization countermeasures, our method succeeds with as few as 150 traces where all existing approaches fail to reach a guessing entropy equal to zero, highlighting its robustness in challenging real-world conditions.

Keywords: Side-Channel Analysis · Blind Side-Channel Analysis · Deep Learning · Unsupervised Trace Labeling · Meta-learning

1 Introduction

Side-channel analysis (SCA) is one of the most potent threats to the security of cryptographic implementations [KJJ99]. By observing physical leakage such as power consumption or electromagnetic radiation during device operation, the attacker can extract secret cryptographic keys even if the cryptographic algorithm is proven theoretically secure. SCA is generally categorized into two major types: profiled SCA and non-profiled SCA. Profiled attacks, including template attacks [BLR12,CRR02], stochastic attacks [CK14,SLP05], and

Licensed under Creative Commons License CC-BY 4.0.
Received: 2026-04-15 Accepted: 2026-07-15 Published: 2026-09-11

the emerging deep learning-based approaches [PWP22,MPP16], exhibit strong performance when a fully controllable clone device identical to the target is accessible. In this scenario, the attacker collects measurements from the clone device to construct a profile, then matches traces from the target device with unknown key but known plaintext/ciphertext to retrieve the secret key. On the other hand, non-profiled attacks, such as correlation power analysis [BCO04], mutual information analysis [VCS09], and more recently, deep learning-based methods that directly optimize a scalar distinguisher over all key guesses without explicit profiling [Tim19, YWQ+25, Yu26], enabling key recovery without a clone device but still exploiting known plaintext/ciphertext to compute intermediate values.

Although both profiled and non-profiled attacks show considerable capability in key recovery, they fundamentally rely on the attacker's ability to obtain known plaintext/ciphertext. This reliance on accessible input/output data underpins most conventional side-channel methods, yet it does not hold in many practical scenarios where such information is unavailable. For example, widely used encryption modes, such as Offset CodeBook (OCB) [TMC+21] restrict direct access to the input/output at the block-level. Similarly, in the key derivation process of the EMV payment standard, full ciphertexts are unrecoverable, and only limited fragments of plaintext may be observable [BSTP21]. These practical limitations create significant challenges for SCA and have motivated the development of specialized attacks against devices with unknown plaintext/ciphertext, known as Blind SCA [CR17, LDLL14, RJBC23, RYJ+25].

Blind SCA follows a three-phase attack workflow. In phase 1, for each key guess, the attacker precomputes the theoretical joint Hamming Weight (HW) distribution of the targeted intermediate values. In phase 2, a labeling method is used to assign HW labels of the target variable to each trace. Then the neural network is trained to further correct these labels to improve performance[1] [LDLL14, CR17, RYJ+25]. In phase 3, this empirical distribution is matched with the theoretical distributions of all key guesses, and the guess yielding the highest likelihood is declared the correct key. The entire attack therefore stands or falls on the quality of phase 2 where an inaccurate labeling method distorts the empirical distribution and causes the correct key guess to be indistinguishable from wrong ones. We note that although Ravi et al. [RJBC23] demonstrated SCA on 32-bit STM32F3 Kyber without known plaintext, their method presupposes a strong attacker possessing a clone device and ground-truth leakage traces for classifier's training where a strictly stronger assumption than the blind setting considered here.

Motivated by the importance of phase 2, a line of research has progressively improved blind labeling methods. Linge et al. [LDLL14] introduced Slicing labeling, which ranks power trace amplitudes in ascending order to approximate HW classes. Clavier et al. [CR17] proposed Variance Analysis (VA) labeling, which exploits the linear relationship assumption between leakage and HW values of intermediate values to derive labels. Both methods are inherently univariate. They estimate the HW of a single intermediate variable from one Points of Interest (PoI), discarding the rich joint information available across multiple PoIs and multiple variables. Rezaeezade et al. [RYJ+25] addressed this limitation with two advances. First, they proposed Multi-point Cluster-based (MC) labeling, which clusters traces using multiple PoIs per variable and models the joint HW distribution across variables simultaneously. Second, they incorporated deep learning into the blind SCA pipeline. The MC-labeled traces are split into a training set and an attack set; a neural network trained on the former is applied to the latter to produce sharper empirical distributions, yielding the first Deep Learning-based Blind SCA (DL-BSCA) framework.

Despite these advances, it only reduces the guessing entropy to zero on unprotected AES implementations. As a result, there is still considerable room for further enhancement. Its efficacy is constrained by two fundamental limitations. First, multivariate analysis induces a large clustering space, and the number of clusters grows exponentially with the number of

[1] This step was introduced by [RYJ+25].

target variables. For instance, targeting three intermediate values in ASCON yields 9^3=729 joint HW classes, while targeting two coefficient groups in Kyber yields 17^2=289 classes, which limits the discriminative power of the Gaussian Mixture Model (GMM) used in MC labeling and increases its sensitivity to noise and outliers, a phenomenon consistent with the well-known curse of dimensionality in high-dimensional density estimation. Second, the labels generated by the MC method are used to train neural networks with uniform sample weighting, implicitly assuming equal label reliability across all traces. However, a trace located deep within a well-separated cluster, which we term it in a Trust Anchor (TA) set, is intuitively far more likely to be correctly labeled than a borderline trace lying equidistant between two cluster centroids. The existing framework ignores this reliability, treating mislabeled and correctly labeled traces equally during training, thereby discarding information inherently captured in the clustering structure.

Contributions. In this paper, we address both limitations and summarize our main contributions as follows:

1. We propose a new DL-BSCA framework Trust Anchor Meta-weight Estimation (TAME), motivated by the observation that labels obtained from clustering-based labeling methods exhibit varying confidence levels. TAME exploits this by constructing a compact validation set from TA, and analyzing a bilevel framework that automatically learns per-sample weights, guided by the validation loss on these anchors.

2. We introduce an efficient unsupervised labeling scheme, called Multi-point Cluster Deep Representation (MCDR) labeling method. By performing clustering in a compact learned feature space rather than in the raw joint HW space, MCDR reduces the impact of noise and outliers while lowering data dimensionality. This effectively mitigates the issue of an excessively large clustering space and further improves attack performance. Crucially, better-separated clusters produced by MCDR yield higher-quality TA set, which in turn strengthens the meta-weight supervision in TAME, making MCDR and TAME mutually reinforcing components of the proposed framework.

3. Our approach achieves strong performance across three standard algorithms (AES, ASCON, and Kyber), as well as under desynchronization countermeasures. On all three datasets, we successfully recover the secret key using at most 8,000 traces, whereas prior methods fail to do so and exhibit a guessing entropy consistently greater than 3. In the desynchronized setting, our method requires only 150 traces to recover the key, while all existing approaches fail. This result highlights the robustness of our approach against desynchronization.

2 Background

2.1 Blind SCA

In a blind SCA scenario, the attacker has no access to known plaintext/ciphertext, and must recover the secret key exclusively from side-channel measurements captured during the target device's execution. This constitutes a considerably more challenging problem compared to conventional SCA. In this work, we adopt the blind SCA framework formalized by Linge et al. [LDLL14], Clavier et al. [CR17], and Rezaeezade et al. [RYJ+25], which consists of three phases as illustrated below.

2.1.1 Phase 1: Pre-computing the theoretical joint distribution

Suppose y is the sensitive intermediate value, and the public variables involved in the operation of y are $m_1, m_2, \ldots, m_{n_b}$, where n_b denotes the number of public variables. The most common analysis target is the output of the AES Sbox, i.e.,$y = \text{Sbox}[m_1 \oplus k^*]$, where m_1 is the plaintext and k^* is the secret key [LDLL14, CR17, RYJ+25][2]. Under the HW leakage model, the joint distribution of $(\text{HW}(m_1), \text{HW}(y))$ is uniquely determined by k^*, making it distinct for each key guess k. Let $(h^*_{m_1}, h^*_y)$ denote the true HW pair of (m_1, y). The attacker therefore enumerates all values of m_1 to count the frequency of each pair $(\text{HW}(m_1), \text{HW}(y))$ for every key guess k, and then normalizes the counts to represent the probability distribution $\Pr((h^*_{m_1}, h^*_y) \mid k)$. Because this computation does not depend on the traces acquired on the target device, it can be precomputed entirely offline.

2.1.2 Phase 2: Labeling traces to obtain the empirical distribution.

The objective of this phase is to assign estimated HW labels to the collected traces, constructing an empirical joint distribution for comparison in Phase 3. This phase consists of three steps.

Step 1: PoI selection. For each targeted variable $v \in \{m_1, y\}$, the attacker identifies the PoIs in the trace that are most strongly correlated with $HW(v)$. Following prior work [LDLL14, CR17, RYJ+25], we adopt the standard assumption that PoIs are located using the Pearson correlation between trace samples and correct $HW(v)$, selecting the feature point with higher absolute correlation. This is the only step that requires access to the public variables and thus constitutes the only non-blind component of the attack.

Step 2: Labeling traces. Using the identified PoIs, a HW label (h_{m_1}, h_y) is assigned to each trace using labeling methods. The accuracy of labeling is crucial, and researchers have proposed numerous methods that can be categorized into two classes: single-point and multi-point labeling.

1. **Single-point labeling.** Both Linge et al. [LDLL14] and Clavier et al. [CR17] assign labels from a single PoI per variable, treating m and y independently.

   (1) **Slicing labeling** [LDLL14]. Linge et al. infer the pair of Hamming weights (h_m, h_y) from signal magnitudes at the chosen PoI, an approach they name Slicing. Their core premise is that lower power dissipation levels at the PoI correspond to a smaller HW of the related intermediate value. Let N stand for the total traces requiring annotation, and l_v represent the power amplitude at the chosen PoI linked to $\text{HW}(v)$, with v representing either m_1 or y. The authors first rank traces in an increasing order according to l_v. The lowest magnitudes are then naturally assigned to $h_v = 0$, followed by the next interval to $h_v = 1$, and so forth. For a B-bit target variable, the HW spans from 0 (all bits cleared) to B (all bits set). To properly distribute traces across each HW class, the authors partition the dataset following the theoretical HW distribution. The fraction of each weight level from 0 to B is given by the binomial coefficient $\binom{B}{h_v}$. Under the uniform distribution hypothesis for cryptographic inputs, $\frac{N}{2^B}\binom{B}{h_v}$ of N traces are expected to exhibit HW h_v for $0 \le h_v \le B$. This strategy is executed individually for h_{m_1} and h_y, yielding the combined label (h_{m_1}, h_y) for every trace.

   (2) **Variance Analysis (VA) labeling** [CR17]. In a similar vein to [CR17], VA labeling also relies on the single appropriate PoI corresponding to $\text{HW}(v)$,

[2]In the following phases, we also take this target as an example for illustration.

$v \in \{m_1, y\}$. Rather than employing slicing for trace annotation, they introduce a linear regression scheme to assign labels. Initially, the authors model the noisy leakage at a feature point as $l_v = \alpha_v \mathrm{HW}(v) + \beta_v + \varepsilon_v$, where α_v and β_v denote unknown constants, ε_v represents Gaussian noise. Accordingly, the variance can be expressed as $\mathrm{Var}(l_v) = \alpha_v^2 \mathrm{Var}(\mathrm{HW}(v)) + \mathrm{Var}(\varepsilon_v)$. The value $\mathrm{HW}(v)$ follows a binomial distribution $\mathcal{B}(B, p)$, where p is the probability that a bit equals 1. Its variance is therefore given by $Bp(1-p)$. Given that bits are uniformly distributed between 0 and 1, $p = \frac{1}{2}$ and $\mathrm{Var}(\mathrm{HW}(v)) = \frac{B}{4}$. The coefficients α_v and β_v can be derived as:

$$\alpha_v = \sqrt{4(\mathrm{Var}(l_v) - \mathrm{Var}(\varepsilon_v))/B},\ \beta_v = \mathbb{E}(l_v) - 4\alpha_v.$$

From these parameters, the estimated HW is computed as $h_v = \frac{l_v - \beta_v}{\alpha_v}$, which yields the paired labels (h_{m_1}, h_y) for each trace.

2. **Multi-point labeling.** Single-point methods rely on one PoI per variable and label each variable independently, discarding the correlation structure across variables. Rezaeezade et al. [RYJ+25] overcame this restriction and proposed the Multi-point Cluster-based (MC) labeling. It is executed in two stages:

   (1) **Produce clusters.** It concatenates multiple PoIs per variable into a joint feature vector and applies GMM to produce clusters in this multivariate space. For example, we select 50 PoIs for $\mathrm{HW}(m_1)$ as PoI_{m_1} and 50 PoIs for $\mathrm{HW}(y)$ as PoI_y, yielding truncated traces with 100 feature points. Then, truncated traces consisting of 100 feature points are fed into the GMM clustering algorithm. The number of clusters is determined by two factors: the bit-width B of each target variable and the count n_b of variables under investigation. As a result, the total number of clusters is set to $M = (B+1)^{n_b}$. Rather than performing separate clustering using samples from PoI_{m_1} and PoI_y individually, they combine both sets of feature points and groups traces that share identical values of $(\mathrm{HW}(m_1), \mathrm{HW}(y))$ into a single cluster.

   (2) **Provide labels.** Clustering groups traces according to similarity, but the resulting clusters remain unlabeled. Therefore, each cluster C must be associated with a label $(Y_C^{(m_1)}, Y_C^{(y)})$ corresponding to (h_{m_1}, h_y). The labeling procedure is performed as follows. First, they compute the centroid of each cluster. Let CT_C denote the center of cluster C, obtained by averaging all traces within the cluster over each PoI. Formally, $CT_C[i] = \frac{1}{N_C} \sum_{t=0}^{N_C-1} \mathrm{trace}_C[t, i]$, where N_C is the number of traces in cluster C. Second, the centroid is partitioned according to PoI_{m_1} and PoI_y, yielding $CT_C^{\mathrm{PoI}_{m_1}}$ and $CT_C^{\mathrm{PoI}_y}$. Considering all M clusters, we form the sets $CT^{\mathrm{PoI}_{m_1}} = \{CT_{C_j}^{\mathrm{PoI}_{m_1}}\}_{j=1}^{M}$ and $CT^{\mathrm{PoI}_y} = \{CT_{C_j}^{\mathrm{PoI}_y}\}_{j=1}^{M}$. The following describes the labeling process for $CT^{\mathrm{PoI}_{m_1}}$, and the same procedure applies to CT^{PoI_y}. The final step involves applying Slicing labeling to the cluster centers of the M clusters corresponding to each PoI. Since the variable m_1 is associated with multiple PoIs, a weighted summation strategy is adopted to derive a single unified label. Specifically, the final label is given by

$$Y_{C_j}^{(m_1)} = \arg \max_{h \in \{0,\ldots,B\}} \left(\frac{|CT_{C_j}^{\mathrm{PoI}_{m_1,h}}|}{\binom{B}{h}} \right),$$

   where $|CT_{C_j}^{\mathrm{PoI}_{m_1,h}}|$ represents number of feature points in $CT_{C_j}^{\mathrm{PoI}_{m_1}}$ labeled as the Hamming weight h through slicing labeling. The weighting compensates for the imbalance inherent in Hamming weight distributions, where mid-range values occur more frequently than extreme ones. By assigning weights proportional

to $\binom{B}{h}^{-1}$, the influence of underrepresented Hamming weights is enhanced, preventing dominant values from biasing the final decision.

Step 3: Deep learning training. Traditional blind SCA methods [LDLL14, CR17] often assume a monotonic relationship between Hamming weights and power consumption and use this assumption to generate labels. In practice, however, such relationships are frequently obscured in raw traces. In contrast to traditional blind SCA, which directly uses labels produced by labeling methods to compute the empirical distribution, Rezaeezade et al. [RYJ+25] leverage the strong generalization capabilities of neural networks. They split trace data into a training set N_t and an attack set N_a, derive labels on the training set N_t in Step 2, train a neural network accordingly, and then infer labels for the attack set N_a using the trained model to construct the empirical distribution.

2.1.3 Phase 3: Distribution comparison and key recovery

A variety of approaches have been proposed to measure the similarity between empirical distributions and their corresponding theoretical joint distributions. In [LDLL14], several metrics were explored, including the χ^2 distance, inner product, and harmonic mean. Alternatively, Le Bouder [LB14] advocated the use of a maximum likelihood criterion. A comparative study in [CR17] showed that the maximum likelihood approach consistently achieves superior performance, and we therefore adopt it in the following.

Let the estimated pair obtained from a labeling method be $(h_{m_1}, h_y) = (h^*_{m_1} + \varepsilon_m, h^*_y + \varepsilon_y)$, where ε_{m_1} and ε_y are Gaussian noise variables with standard deviations σ_{m_1} and σ_y, respectively. According to Bayes' rule, the posterior probability of a key k given a single observation (h_{m_1}, h_y) can be expressed as

$$Pr(k|(h_{m_1}, h_y)) = \frac{Pr((h_{m_1}, h_y)|k)Pr(k)}{Pr((h_{m_1}, h_y))}.$$

Since the denominator is independent of k, it can be omitted, and we assume a uniform prior over keys. For a sequence of observations $\{(h_{m_1}, h_y)_i\}_{i=1}^N$, the posterior can be computed recursively. Using the law of total probability, the likelihood term can be written as

$$\Pr((h_{m_1}, h_y) \mid k) = \sum_{h^*_{m_1},\, h^*_y} \Pr\big((h_{m_1}, h_y) \mid (h^*_{m_1}, h^*_y)\big) \Pr\big((h^*_{m_1}, h^*_y) \mid k\big)\,.$$

Here, the second factor corresponds to the theoretical joint distribution, and the first captures the noise model. Assuming independent Gaussian perturbations, it can be expressed as

$$\Pr\big((h_{m_1}, h_y) \mid (h^*_{m_1}, h^*_y)\big) = \frac{1}{2\pi\sigma_m\sigma_y} \exp\left(-\frac{(h_{m_1} - h^*_{m_1})^2}{2\sigma^2_{m_1}} - \frac{(h_y - h^*_y)^2}{2\sigma^2_y}\right).$$

In practice, the observed values (h_{m_1}, h_y) are obtained through the labeling method and are subsequently used as empirical inputs for likelihood evaluation.

2.2 Deep temporal clustering representation

Deep Temporal Clustering Representation (DTCR) [MZLC19] is a joint representation learning and clustering framework originally proposed for time-series data. Its main idea is to learn an encoder that maps variable-length sequences into a compact, fixed-dimensional embedding space in which K-means clustering can be performed effectively. Unlike two-stage pipelines that first learn representations independently and then cluster them, DTCR trains the encoder end-to-end by simultaneously optimising three complementary objectives.

The first objective is a reconstruction loss based on a denoising sequence-to-sequence autoencoder. The encoder maps an input sequence to a fixed-length representation vector, and a decoder reconstructs the original clean sequence from this vector. This reconstruction objective encourages the encoder to capture the essential structure of the time-series while suppressing noise, yielding a compact and noise-robust embedding.

The second objective is a K-means-friendly clustering loss derived from a spectral relaxation of the K-means objective. Rather than alternating between representation learning and a separate clustering step, this loss is incorporated directly into the training objective, shaping the geometry of the embedding space so that representations of sequences belonging to the same cluster are drawn together and can be separated by hyperplanar boundaries consistent with K-means assignments.

The third objective is a discriminative loss that exploits synthetically generated negative samples. Fake sequences are constructed by interpolating between real training samples in the input space; the encoder is then trained to distinguish genuine representations from those of the fake sequences. This contrastive signal sharpens inter-class boundaries in the embedding space, enhancing the discriminative power of the learned representations beyond what reconstruction alone can provide.

The three losses are optimised jointly, so that the encoder simultaneously denoises, clusters, and discriminates. At inference time, sequences are encoded into their fixed-length representations and assigned to clusters through standard K-means, yielding cluster labels that reflect the underlying temporal structure of the data.

3 The TAME framework

The TAME framework is motivated by the observation that clustering-based labeling produces labels of inconsistent reliability, while the clustering geometry contains useful confidence information that uniform-weight training discards entirely. Its workflow in the DL-BSCA analysis framework is illustrated in Figure 1. TA selection extracts high-confidence traces near cluster centroids to form a validation set, as illustrated in Section 3.2.1. Meta-weight Estimation (ME) then performs bilevel optimization to learn sample-wise weights guided by validation loss on these anchors, which is detailed in Section 3.2.2, automatically upweighting reliable samples and downweighting noisy ones. Two variants extend the core framework. In Section 3.3, MCDR labeling substitutes MC labeling's GMM with deep temporal clustering in a learned low-dimensional space, mitigating exponential cluster growth and enhancing TA quality for meta-weight estimation. In Section 3.4, TAME-GT reuses the small set of ground-truth labeled traces reserved for PoI selection as the validation set, yielding a performance upper bound that measures how closely blind TAME approximates ideal semi-supervision.

3.1 Threat model

We adopt the same threat model as the blind SCA formulations described in [LDLL14, CR17, RYJ+25]. The attacker possesses no prior information about the processed data, such as plaintext/ciphertext, and exclusively depends on side-channel measurements to recover the secret key. This constitutes a significantly more challenging setting than conventional SCA attacks, in which each trace is paired with a known plaintext/ciphertext. Furthermore, consistent with the standard blind SCA pipeline, we assume the attacker can identify the most representative PoIs, which is determined by evaluating the correlation between traces and the true HW values of the targeted intermediate variables.

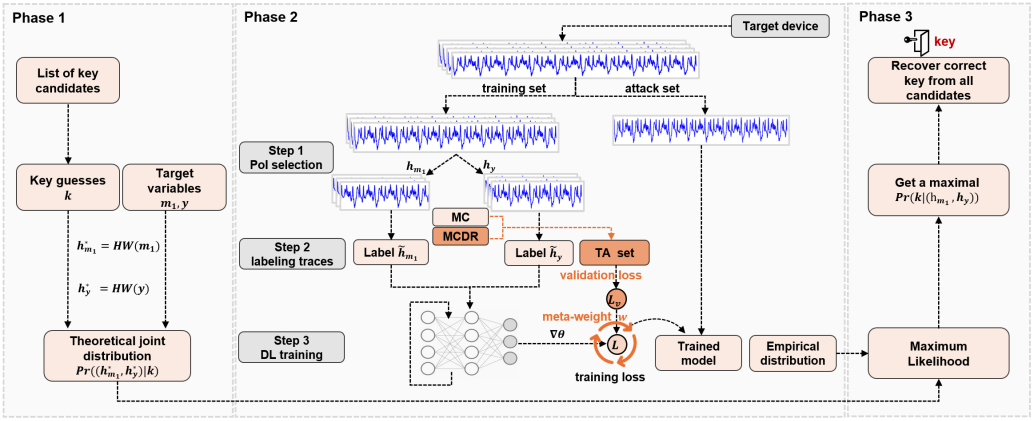


Figure 1: Analysis framework of DL-BSCA. **Spanish orange blocks** denote components proposed in this work: MCDR labeling (Section 3.3), TA selection (Section 3.2.1), and Meta-weight Estimation (Section 3.2.2). Light orange blocks denote existing components from prior work [LDLL14, CR17, RYJ+25]: Phase 1 pre-computation, PoI selection, standard DNN training, and Phase 3 distribution comparison.

3.2 Methodology

The design of TAME is driven by two concrete research challenges in existing DL-BSCA:

1. **Exponential growth of the clustering space.** Multivariate clustering assigns each trace to one of $M = (B+1)^{n_b}$ joint Hamming weight classes, which grows exponentially with the number of targeted variables n_b. For ASCON this yields 9^3=729 clusters; for Kyber, 17^2=289. Fitting a GMM across hundreds of sparse, high-dimensional clusters severely limits discriminative power and increases noise sensitivity, which is a direct consequence of the curse of dimensionality. MCDR labeling (Section 3.3) addresses this challenge.

2. **Uniform treatment of pseudo-labeled traces.** The existing framework assigns equal loss weight to every pseudo-labeled trace, ignoring the confidence gradient implicit in the clustering geometry: a trace deep within a compact cluster is far more likely to be correctly labeled than a borderline trace equidistant between two centroids. Treating all samples equally allows mislabeled traces to corrupt the gradient signal. TA selection and Meta-weight Estimation (Sections 3.2.1–3.2.2) address this challenge.

3.2.1 Part 1: TA selection

The Hamming weight labels derived from the labeling method are referred to as pseudo-labels. Existing DL-BSCA method divides traces into training and attack sets. They adopt the MC labeling method to assign labels to the training set, train a neural network using these pseudo-labels, and then let the model generate final labels on the attack set. During training, the network treats all pseudo-labels as equally reliable, which is reflected in the computation by assigning equal loss weights to each sample. However, the accuracy of pseudo-labels generated by labeling is rather low, often below 10%, indicating that the network can be easily misled by incorrect labels. Meanwhile, the MC method employs GMM clustering analysis, which not only provides the class of each trace but also the confidence of its class assignment, yet this information is not fully exploited in subsequent stages.

Within the GMM framework, the soft posterior probability of a trace belonging to its assigned component is maximised at the component centroid and decreases as the trace moves toward the boundary between adjacent components [BN06]. Consequently, a trace residing deep within a compact, well-separated cluster is an unambiguous representative of its Hamming weight class and is almost certainly correctly labeled. By contrast, a trace lying near the boundary between two adjacent clusters could plausibly belong to either class; its assigned label is therefore untrusted and potentially wrong. Uniform sample weighting in neural network training ignores this confidence gradient, allowing mislabeled boundary traces to corrupt the gradient signal on an equal footing with high-confidence core samples. We address this by identifying a subset of TA, traces whose cluster membership is most certain, and using them as a validation set to guide training process described in Section 3.2.2. Next, we describe how to select the set of TA from the GMM clustering algorithm.

TA Selection for MC labeling. TA selection is detailed in Algorithm 1. Given the GMM model fitted during MC labeling, the TA set is constructed as follows. For each cluster $C \in \{1, \ldots, M\}$, let $\mathcal{I}_C = \{i : \text{GMM}(\text{trace}_i) = C\}$ be the index set of training traces assigned to that cluster. The proximity of trace i to its cluster centroid is measured by the Euclidean distance $d_i = \|\text{trace}_i - \text{CT}_C\|_2$, where the distance is computed in the original, non-standardised feature space, the same as space in which the GMM was fitted, so that centroid coordinates and trace representations are directly comparable.

A per-cluster quota $\kappa = \lceil N_\text{v}/M \rceil$ is then allocated, where N_v is the target validation set size. From each cluster k, the $\min(\kappa, |\mathcal{I}_k|)$ traces with the smallest d_i are selected as candidate TA, with the cap $|\mathcal{I}_k|$ preventing failure for sparsely populated clusters. All per-cluster candidates are pooled into a common set; if the pool exceeds N_v, a random subsample of size N_v is drawn to form the final set $\mathcal{D}_{val}$, introducing controlled stochasticity while preserving the centroid-proximity guarantee.

The rationale for per-cluster allocation rather than a global top-N_v selection is class-coverage balance. Clusters corresponding to extreme Hamming weights (e.g., h=0 or h=B) contain fewer traces yet tend to be more geometrically compact, so a global selection would systematically over-represent them while omitting traces from mid-range, sparser clusters. Since the bilevel optimisation in Section 3.2.2 adjusts per-sample weights by minimising the validation loss, an imbalanced $\mathcal{D}_{val}$ would skew the learned weights toward over-represented classes, degrading attack performance on the remainder. The per-cluster quota enforces that every joint Hamming weight class contributes equally to the validation signal.

The resulting set $\mathcal{D}_{val}$ and its associated pseudo-labels serve as the validation set for the bilevel optimisation in Section 3.2.2. Crucially, the entire selection procedure is unsupervised. It requires only the cluster traces and centroid coordinates produced by the GMM fit, with no access to ground-truth Hamming weight labels. This preserves the fully blind nature of the attack. The pseudo-labels carried by TA are themselves produced by the same GMM clustering, so the validation set is not label-noise free. It is simply a high-confidence subset of the pseudo-labeled training data.

3.2.2 Part 2: Meta-weight estimation through bilevel optimization

Once the TA set $\mathcal{D}_{val}$ is built as outlined above, the key objective is to use this set to enhance neural network training on the training dataset N_t. The primary challenge lies in the inconsistent quality of pseudo-labels generated by MC labeling. Some training samples carry reliable labels whereas others do not, and the network cannot identify the difference in advance. Training with equal sample weighting, as applied in [RYJ+25], regards every sample as equally trustworthy. This method allows mislabeled traces to disrupt gradient computation throughout the training phase.

Algorithm 1 TA selection

Input: Training set $\mathcal{D}_{tr} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N_t}$, cluster assignments $\{c_i\}_{i=1}^{N_t}$ with $c_i \in \{1, \ldots, M\}$, cluster centres $\{CT_C\}_{C=1}^{M}$ in representation space $\mathcal{Z}$, requested validation size N_v, random seed s

Output: Validation set $\mathcal{D}_{val}$

$\kappa \leftarrow \lceil N_v / M \rceil$ ▷ budget per cluster

$\mathcal{C} \leftarrow \emptyset$ ▷ initialize candidate pool

for $C = 1$ **to** M **do**

  $\mathcal{I}_C \leftarrow \{i : c_i = C\}$

  $d_i \leftarrow \|\mathbf{x}_i - CT_C\|_2 \quad \forall\, i \in \mathcal{I}_C$ ▷ distance to cluster centre

  $\mathcal{C} \leftarrow \mathcal{C} \cup \textsc{TopK}(\mathcal{I}_C, \{d_i\}, \min(\kappa, |\mathcal{I}_C|))$ ▷ keep nearest samples

end for

if $|\mathcal{C}| \geq N_v$ **then**

  $\mathcal{V} \leftarrow \textsc{RandomSample}(\mathcal{C}, N_v, \text{seed} = s)$

else

  $\mathcal{V} \leftarrow \mathcal{C}$

end if

return $\mathcal{D}_{val} \leftarrow \{(\mathbf{x}_i, y_i) : i \in \mathcal{V}\}$

We introduce a **bilevel optimization strategy** in which per-sample weights are learned under the guidance of the TA validation set. The outer level optimizes these weights to minimize the validation loss on $\mathcal{D}_{val}$. The inner level trains the network under the resulting weighted objective. This design naturally reflects the intended behavior. A sample with an accurate pseudo-label will lower the validation loss when assigned greater weight and thus be allocated a higher importance value. A mislabeled sample will produce the opposite effect and receive reduced weight automatically.

Bilevel optimizations have been explored for meta-learning in supervised settings [KL17, SLGH21, BCC+20, RZYU18, WRLG18, SXY+19], where in the inner loop stage the model fits a specific task using training samples and in the outer loop stage an objective function for the outer loop is optimized. In a blind SCA setting, no ground-truth labels are available, so a validation set cannot be reasonably assumed. TAME addresses this difficulty by building the validation set from TA. These samples are recognized as high-confidence candidates based only on clustering geometry, making the bilevel optimization applicable for the first time in a blind SCA scenario. Algorithm 2 gives the complete procedure. Second, we introduce a weight normalization strategy to handle unreliable meta-gradients. In prior bilevel optimization schemes, when the meta-gradient norm falls below a threshold ($S_t = 0$), the update is skipped entirely by assigning zero weights to the mini-batch. In contrast, our approach reverts to a uniform-weight update instead of discarding the batch. This design choice is motivated by the fact that our validation set only contains higher-confidence pseudo-labels. During the early stages of training, when pseudo-label quality is still limited and meta-gradients are often uninformative, skipping updates can significantly hinder convergence. By falling back to uniform weighting, we ensure that training continues in a stable manner.

Bilevel optimization. Let $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N_t}$ be training set and $\{(\mathbf{x}_i^v, y_i^v)\}_{i=1}^{N_v}$ be the TA validation set with $N_v \ll N_t$. Let $\Phi(\mathbf{x}, \theta)$ denote the neural network with parameters θ, and let $f_i(\theta) = \mathcal{L}(\Phi(x_i, \theta), y_i)$ and $f_i^v(\theta) = \mathcal{L}(\Phi(x_i^v, \theta), y_i^v)$ denote the per-sample losses on the training and validation sets, respectively, where $\mathcal{L}$ is the loss function. The meta-weight

estimation problem is formulated as the following bilevel optimisation:

$$\omega^* = \arg \min_{\omega, \omega \geq 0} \frac{1}{N_v} \sum_{i=1}^{N_v} f_i^v(\theta^*(\omega)) \,, \tag{1}$$

$$\theta^*(\omega) = \arg\min_{\theta} \sum_{i=1}^{N_t} \omega_i \, f_i(\theta). \tag{2}$$

The inner level defined in Equation (2) trains the network using a weighted loss function. The outer level shown in Equation (1) selects the weights ω^* that minimize the validation loss over the TA. The constraint $\omega_i \geq 0$ avoids degenerate solutions where harmful samples might be exploited through the minimization of their negated training loss. The nested structure shared by Equation (1)–(2) leads to extremely high computational cost for exact solutions. This expense arises because a complete inner optimization cycle must be executed for each outer gradient update.

Algorithm 2 Meta-weight Estimation (ME)

Input: Training set $\mathcal{D}_{tr} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N_t}$, validation set $\mathcal{D}_{val} = \{(\mathbf{x}_j^v, y_j^v)\}_{j=1}^{N_v}$, network $f_{\boldsymbol{\theta}}$, learning rate η, mini-batch size n, total iterations T
Output: Trained parameters $\boldsymbol{\theta}^*$

for $t = 1$ **to** T **do**
  $\mathcal{N} \leftarrow \textsc{RandomSample}(\mathcal{D}_{tr}, n)$
  $\boldsymbol{\epsilon} \leftarrow \mathbf{0} \in \mathbb{R}^n$
  $\ell_i \leftarrow \mathcal{L}(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i), \quad \mathcal{L}_{tr}(\boldsymbol{\epsilon}) \leftarrow \sum_{i=1}^{n} \epsilon_i \, \ell_i$
  $\hat{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} - \eta \, \nabla_{\boldsymbol{\theta}} \mathcal{L}_{tr}(\boldsymbol{\epsilon})$ ▷ simulated meta-update
  $\mathcal{L}_{val} \leftarrow \frac{1}{N_v} \sum_{j=1}^{N_v} \mathcal{L}(f_{\hat{\boldsymbol{\theta}}}(\mathbf{x}_j^v), y_j^v)$
  $\mathbf{g} \leftarrow \nabla_{\boldsymbol{\epsilon}} \mathcal{L}_{val}(\hat{\boldsymbol{\theta}})$ ▷ differentiate through meta-update
  $\tilde{w}_i \leftarrow \max(-g_i, \, 0) \quad \forall \, i$
  if $\sum_i \tilde{w}_i > 0$ **then**
    $w_i \leftarrow \tilde{w}_i \, / \, \sum_j \tilde{w}_j$
  else
    $w_i \leftarrow 1/n \quad \forall \, i$
  end if
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla_{\boldsymbol{\theta}} (\sum_{i=1}^{n} w_i \, \ell_i)$
end for
return $\boldsymbol{\theta}$

Meta-weight estimation. To make the bilevel optimization tractable within a single training loop, we introduce a meta-weight estimation. First, we consider perturbing the weighting by ε_i for each training example in the mini-batch $\{(x_i, y_i)\}_{i=1}^{n}$. Instead of solving the inner optimization problem until full convergence, we approximate the process with a single gradient update step. For a perturbation vector $\varepsilon \in \mathbb{R}^n$ initialized to $\mathbf{0}$, we define the one-step perturbed update as

$$\hat{\theta}_{t+1}(\varepsilon) = \theta_t - \alpha \nabla_\theta \left[\sum_{i=1}^{n} \varepsilon_i \, f_i(\theta_t) \right],$$

where α is the learning rate. We can then look for the optimal ε^* that minimizes the validation loss f^v locally at step t:

$$\varepsilon_t^* = \arg\min_{\varepsilon} \frac{1}{N_v} \sum_{i=1}^{N_v} f_i^v(\theta_{t+1}(\varepsilon)).$$

As this procedure can still be computationally expensive, we further approximate ε_t^* by performing a single gradient descent step on a mini-batch of validation samples $\{(x_i^v, y_i^v)\}_{i=1}^m$, from which we obtain $u_{i,t}$. We set $G_t(\varepsilon) = \frac{1}{m}\sum_{j=1}^m f_j^v(\hat{\theta}_{t+1})(\varepsilon)$,

$$u_{i,t} = -\eta \frac{\partial G_t}{\partial \varepsilon_i} = -\eta(\nabla_\theta G_t(\varepsilon))^T \frac{\partial \theta_{t+1}(\varepsilon)}{\partial \varepsilon_i} = \eta\alpha < \nabla_\theta \bar{f}^v(\theta_t), \nabla_\theta f_i(\theta_t) > . \tag{3}$$

From the above formulation, we observe that if the gradient of training sample i aligns with the validation gradient, the sample should be assigned a higher weight. If the gradients point in opposite directions, the sample is likely to be noise or mislabeled data.

Weight normalisation and blind-SCA fallback. We then rectify and normalize the final per-sample weight for the current step. Setting $\tilde{\omega}_{i,t} = \max(u_{i,t}, 0)$ and $S_t = \sum_{i=1}^n \tilde{\omega}_{i,t}$, the normalised weight is

$$\omega_{i,t} = \begin{cases} \tilde{\omega}_{i,t} \,/\, S_t & \text{if } S_t > 0, \\ 1/n & \text{if } S_t = 0. \end{cases}$$

The ℓ_1 normalisation in the $S_t > 0$ branch stabilises training by keeping the effective learning rate constant regardless of weight magnitudes, and it cancels the meta learning rate η as a free hyperparameter. When $S_t = 0$, we interpret that the TA validation set shows no preference within the current mini-batch. This indicates that the meta-gradient signal lacks useful information. Directly discarding such batches would wastefully suppress potentially useful training signals. In the early stages of training, pseudo-label quality is low and the network has not yet developed discriminative capabilities. Frequent discarding in this period would often hinder convergence progress. Instead, reverting to $\omega_{i,t} = 1/n$ restores the uniform-weight update scheme presented in [RYJ+25]. This design ensures that TAME framework degrades gracefully to the baseline method whenever the meta-gradient signal is uninformative.

3.3 MCDR labeling and its TA selection

The MC labeling method performs GMM clustering in the joint PoIs space. The number of clusters is set to $M = (B+1)^{n_b}$, which represents the total number of distinct joint Hamming weight classes over all n_b targeted variables. This value grows exponentially with n_b. Two 8-bit AES variables result in 81 clusters, while targeting three ASCON variables generates 9^3=729 clusters and two groups of 16-bit Kyber coefficients produce 17^2=289 clusters. Fitting a GMM under such conditions is unreliable due to two mutually reinforcing factors. First, the exponential growth of M disperses available training traces increasingly sparsely across clusters. For example, with 729 clusters and 80,000 traces, each cluster receives only approximately 110 samples on average, which leads to statistically unstable covariance estimation [MP00]. Second, GMM requires estimating a $d \times d$ covariance matrix for each component in a d-dimensional space. In the example mentioned above, estimating full covariance matrices for a 100-dimensional PoI feature vector involves around 3,600,000 parameters. In high-dimensional spaces, all pairwise distances tend to concentrate near a common value. This behavior reduces the discriminative power of Gaussian density estimates and represents a well-known effect of the curse of dimensionality [BN06].

We address this limitation by introducing **Multi-point Cluster Deep Representation (MCDR) labeling**, which replaces the GMM in the raw high-dimensional feature space with clustering performed in a learned low-dimensional representation space. Specifically, MCDR substitutes the GMM step of MC labeling with the Deep Temporal Clustering Representation (DTCR) method [MZLC19], a technique originally developed for time-series clustering that jointly optimises three objectives: (1) a reconstruction loss derived from a denoising sequence autoencoder, which reduces measurement noise and generates a compact embedding for each trace; (2) a spectral relaxation loss for K-means, which adjusts the geometric structure of the embedding space to support clustering and gather traces belonging to the same joint Hamming weight class; and (3) a discriminative loss that contrasts genuine traces with artificially corrupted samples, which sharpens the separation between different classes in the embedding space. The resulting encoder maps each trace to a low-dimensional representation vector [3], on which standard K-means clustering is then applied with the same $(B+1)^{n_b}$ components as in MC labeling. Cluster label assignment follows the same centroid-averaging and Slicing-based voting procedure as MC labeling.

TA selection for MCDR labeling. The TA selection strategy for MCDR follows the same balanced per-cluster design described in Section 3.2.1 for MC labeling, but with a critical adaptation: distances are measured in the DTCR representation space rather than the original PoI feature space. Concretely, after K-means clustering on the encoder representations, the distance of each training trace i to its assigned cluster centroid is computed as $d_i = \|\mathbf{h}_i - \boldsymbol{c}_{k(i)}\|_2$, where $\mathbf{h}_i$ is the DTCR representation of trace i and $\boldsymbol{c}_{k(i)}$ is the K-means centroid of the assigned cluster. The per-cluster quota $\kappa = \lceil N_{\mathrm{v}}/M \rceil$, candidate pooling, and final random subsampling then proceed identically to TA selection for MC labeling case.

The rationale for computing distances in the representation space instead of the original feature space lies in its consistency with the clustering procedure. DTCR clustering is defined and optimized in the representation space, so centroid proximity in that space is the natural and correct measure of cluster membership certainty. Using raw PoI distances instead would be inconsistent, as two traces may be close in PoI space yet far apart in representation space if the encoder has learned to emphasise features orthogonal to raw amplitude. By grounding TA selection in the same space where clustering is performed, MCDR and its TA strategy form a fully unified framework.

3.4 TAME-GT: ground-truth anchor variant

As established in Section 2.1.2, the standard blind SCA pipeline requires the attacker to possess a small set of correctly labeled traces for PoI selection. Correlation values between trace samples and ground-truth Hamming weights are computed to locate the most informative PoIs. In the blind setting of TAME, these labeled traces are only used for PoI selection and are then discarded. The TA validation set is subsequently built from pseudo-labeled samples based only on clustering geometry.

The TAME-GT variant replaces the pseudo-labeled TA with ground-truth labels. Formally, if the attacker holds N_v traces with known joint Hamming weight labels $\{(x_i, y_i^*)\}$, these are used directly as $\mathcal{D}_{val}$ in Equation (1), with no modification to the bilevel training procedure.

The TAME-GT variant serves two purposes in this work. First, it provides a performance upper bound for the blind TAME. Since the validation set carries true labels rather than high-confidence pseudo-labels, the meta-weight supervision signal is maximally trusted, and the resulting attack performance represents the best that the bilevel training strategy can achieve given the same network architecture and training data. Second,

[3] This low-dimensional representation size is a hyperparameter, set to 400 in the following experiments.

it quantifies the gap attributable to TA quality. The performance difference between TAME-GT and blind TAME isolates the effect of replacing ground-truth validation labels with pseudo-labels derived from clustering. A small gap confirms that TA selection is an effective substitute for ground-truth supervision; a large gap would indicate that pseudo-label quality is the limiting factor.

We emphasize that TAME-GT does not correspond to a weaker threat model than blind SCA. The total label budget available to the attacker remains unchanged. Instead, it reflects a different allocation of the same limited set of true labels. Labels used only for PoI selection in the blind setting are also employed for validation-guided training in TAME-GT. This reallocation supports the interpretation of TAME-GT results as a well-grounded performance upper bound rather than an idealized oracle.

4 Experimental Results

4.1 Experimental setup

Networks and training. We randomly search for 100 models from the parameter search space in Table 1 and Table 7. The comparison between our method and the baseline [RYJ+25] is based on the results corresponding to the best-performing model selected from the 100 models respectively. 50 PoIs are considered for each variable, where these parameters are determined according to the settings described in baseline research [RYJ+25]. Their experimental results indicate that the application of dropout as a regularization technique has no decisive impact on final attack outcomes, so we do not employ dropout in all experiments of this work and pay attention to more structural changes. We also adopt two types of network architectures including Convolutional Neural Networks (CNNs) and Multi-Layer Perceptrons(MLPs). The categorical cross entropy function is used as the loss function for neural networks. We also note that labels,i.e., (h_{m_1}, h_y), are stored in the form of a tuple $(B+1) * h_{m_1} + h_y$ in related computations. Within the TAME framework, the TA set employs a small validation set. In our experiments we set $N_v \in \{5, 10, ..., 200\}$, which means the maximum number of samples in the TA set is 200, a very small quantity for scenarios with unknown plaintexts and ciphertexts. We provide the reproduction code[4].

Evaluation metrics. We adopt two complementary metrics to evaluate attack performance. Guessing Entropy (GE) [SMY09] measures the average rank of the correct key guess across 100 independent attack runs, computed after all available attack traces have been consumed. A method is considered to have failed whenever its final GE exceeds zero, as the correct key remains indistinguishable from competing key guesses. For methods that achieve GE = 0, we additionally report NTGE, defined as the number of attack traces required to first bring GE to zero. A lower NTGE indicates a more efficient attack.

Datasets. In the work of [RYJ+25], the analysis targets include AES-128 and desynchronized AES-128 implemented on 8 bit AVR XMEGA devices, as well as the Kyber and ASCON algorithms implemented on 32-bit STM32F3 ARM Cortex-M platform. We further compute SNR values for the three algorithms. Since the labeling method operates on 50 PoIs for each intermediate variable, we report the average SNR over these 50 PoIs. The results for each intermediate values are $(0.6, 0.1)$ for AES, $(9.6, 10.2)$ for Kyber[5], and $(0.03, 0.1, 0.08)$ for ASCON. Therefore, we argue that the datasets used in [RYJ+25] exhibit higher noise levels than those considered in earlier blind SCA studies [LDLL14, CR17], which targeted unprotected AES implementations on 8-bit AVR devices. The DNN

[4] https://github.com/jintongyucrypto/TAME-A-Deep-Learning-based-Blind1Side-Channel-Analysis-Framework.git

[5] Although the Kyber dataset exhibits higher numerical SNR values, it should be noted that Kyber involves a significantly larger search space (q=3329) than the other two algorithms.

approach proposed in baseline work [RYJ+25] yields attack results that are significantly better than those of traditional methods. We observe that key recovery is not achieved on the experimental datasets used in that study which indicates room for further improvement. Therefore, this work adopts the same datasets and attack points as those used in [RYJ+25].

Table 1: Random search space for MLP

Hyperparameter	Search space
Architecture	
Number of hidden layers	$\{2, 3, 4, 5, 6, 7\}$
Number of neurons per layer	$\{10, 30, 50, 70, 90, 120, 150, 200, 250, 300, 400, 500\}$
Learning hyperparameters	
Weight initialization	{random_uniform, glorot_uniform, he_uniform}
Activation function	{relu, selu, elu, tanh}
Batch size	$\{128, 256, 512\}$
Learning rate	$\{10^{-3},\ 5\times10^{-4},\ 10^{-4},\ 5\times10^{-5},\ 10^{-5}\}$
Optimizer	Adam
Epochs	100

4.2 Main results

We present attack results obtained on AES, Kyber, and ASCON datasets to compare our method with existing approaches in Table 2. Overall, the proposed TAME framework combined with MCDR labeling achieves the best performance across all experiments and successfully recovers the secret key when used with an MLP network. We first focus on improvements to labeling strategies. Existing works concentrate on computing empirical distributions of target variables from leakage traces. A central task in this stage is to generate high-quality pseudo-labels. Researchers have proposed several labeling techniques including Slicing, VA, and MC labeling. We also introduce a new labeling method named MCDR. Attack results for these four methods appear in the first section of Table 2. MCDR delivers the best performance among them but still fails to recover the secret key.

Table 2: Performance comparison of all methods and their combinations.

Datasets	AES			Kyber			ASCON		
	Without DNN	MLP	CNN	Without DNN	MLP	CNN	Without DNN	MLP	CNN
Slicing [LDLL14]	GE=218	-	-	GE=1242	-	-	GE=77	-	-
VA [CR17]	GE=223	-	-	GE=2415	-	-	GE=77	-	-
MC	GE=79	-	-	GE=1146	-	-	GE=45	-	-
MCDR	GE=26	-	-	GE=195	-	-	GE=47	-	-
Slicing+DNN	-	GE=49	GE=109	-	GE=1178	GE=1230	-	GE=74	GE=64
VA+DNN	-	GE=122	GE=110	-	GE=1756	GE=1719	-	GE=63	GE=75
MC+DNN [RYJ+25]	-	GE=15	GE=23	-	GE=423	GE=336	-	GE=38	GE=65
MCDR+DNN	-	GE=8	GE=7	-	GE=188	GE=171	-	GE=3	GE=3
MC+TAME	-	GE=10	GE=6	-	GE=81	GE=157	-	GE=9	GE=13
MCDR+TAME (this work)	-	**660**	GE=3	-	**5546**	GE=1	-	**7605**	GE=1

We next discuss improvements to the pseudo-label refinement framework. Rezaeezade et al. introduced a DNN framework that splits traces into training and attack sets [RYJ+25]. Labeling methods are used to assign pseudo-labels to the training set for network training. The trained model is then used to infer final labels on attack set for key recovery. Since the DNN framework operates independently of the underlying labeling strategy, we apply all four labeling methods within the DNN pipeline in the second section of Table 2. We observe that integrating the DNN framework leads to a significant reduction in GE across all setups. This observation confirms the advantages of the DNN-based refinement pipeline. Consistent with the results reported in [RYJ+25], all methods under the DNN framework achieve relatively low guessing entropy values but still fail to recover the secret key.

Building upon the DNN pipeline, we propose the TAME framework. It selects samples with highly reliable labels from clustering outputs as a TA set. Sample-wise learning weights are then adjusted to steer the optimization direction of the model. Slicing and VA labeling cannot capture implicit label confidence, which contributes to their relatively weak performance. In contrast, MC and MCDR are clustering-based methods where the distance to cluster centroids naturally encodes the reliability of assigned labels. We therefore evaluate MC and MCDR under the TAME framework in the third section of Table 2. Experimental results show that both methods achieve stronger performance when combined with TAME than with the baseline DNN framework. This outcome validates the rationality of our design and analysis. Furthermore, we also find MCDR consistently outperforms the other three labeling approaches regardless of whether the DNN/TAME framework is used.

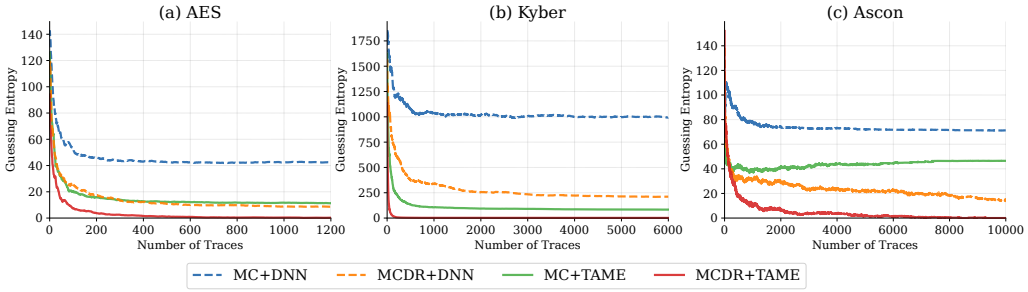


Figure 2: MC vs MCDR: GE evolution comparing MC and MCDR labeling methods under DNN and TAME frameworks on AES, Kyber, and ASCON datasets.

4.2.1 Ablation study on MCDR labeling methods

In this section, we compare different labeling methods through controlled variable experiments. Existing work has demonstrated the superiority of the MC labeling method over Slicing and VA labeling approaches [RYJ+25]. We compare the performance of MC with our MCDR labeling method under both the DNN framework (MC+DNN vs MCDR+DNN) and the TAME framework(MC+TAME vs MCDR+TAME). To ensure a fair comparison, all hyperparameters, including neural network architectures and learning rates for DNN and TAME, are kept consistent.

The evolution of GE across attack traces under one set of model parameters is shown in Figure 2. We draw two key observations from these results. MCDR outperforms MC under both DNN and TAME frameworks. Secondly, the blue line represents the result of MC labeling combined with DNN from existing work [RYJ+25] and maintains the highest GE throughout the attack. This indicates that replacing MC labeling with MCDR leads to performance improvements beyond the current state-of-the-art (SOTA).

4.2.2 Ablation study on TAME

To isolate the contribution of TAME, we compare uniform-weight training in DNN against meta-weight training in TAME under two labeling conditions: MC labeling (MC+DNN vs MC+TAME) and MCDR labeling (MCDR+DNN vs MCDR+TAME). Both the MC method and the MCDR method use identical parameter settings. Parameters for DNN and TAME are selected from the model random search space to obtain the optimal configurations. Figure 3 presents the corresponding GE evolution. On all three datasets and under both labeling approaches, meta-weight estimation consistently yields lower GE compared

with uniform weighting, and improves attack performance by at least 33% compared with the DNN framework. When combined with MCDR, the improvement enables GE=0. Under the MCDR labeling technique, TAME with an MLP backbone successfully recovers the key across all datasets. These results verify that meta-weight estimation offers a complementary and independently effective improvement beyond the quality of labeling, and the better labeling method can further enhance the attack performance of the TAME framework.

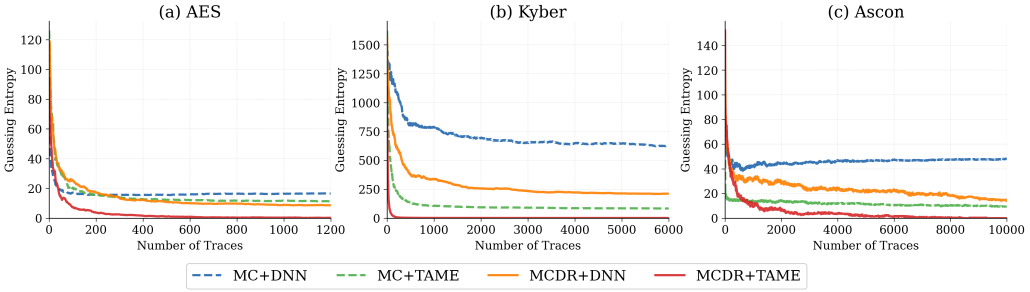


Figure 3: DNN vs TAME: GE evolution comparing DNN and TAME frameworks under two labeling methods on AES, Kyber, and ASCON datasets.

Label accuracy predicted by TAME. A key premise of TAME is that TA set carries more reliable pseudo-labels to finish meta-weight optimization. Whether this assumption holds, and whether meta-weight training enables the network to better capture the relationship between traces and the Hamming weights of the target variables, can be directly evaluated through the accuracy on the attack set. To this end, Figure 4 reports the prediction accuracy of Hamming weight labels on attack traces for both the DNN and the TAME framework. For clarity, we measure accuracy separately for each Hamming weight class, using the MCDR labeling method.

The results reveal three key observations. First, incorporating meta-weight training improves the overall label accuracy by more than a factor of two. Second, the accuracy for individual Hamming weight classes increases significantly, from below 15% in the baseline to as high as 60% under TAME. Third, due to the inherent distribution of Hamming weights, non-central classes (e.g., 8 in 8-bit or 6 in 16-bit settings) are particularly difficult to learn. For instance, in a 16-bit setting, always predicting the central class (Hamming weight 8) yields much higher accuracy than predicting extreme classes such as 0. Neural networks tend to exploit this imbalance, leading to biased predictions that do not reflect the underlying leakage information. However, such behavior is not desirable for SCA, as it fails to capture the true relationship between traces and the target variables. Therefore, improving accuracy on non-central Hamming weight classes is crucial. As illustrated in the middle subplot of Figure 4, the baseline DNN exhibits a class-wise accuracy distribution (blue bars) that closely follows the natural Hamming weight distribution. This explains its relatively poor performance on the Kyber dataset, as reflected by GE=188.

4.3 Gap analysis: blind TAME vs semi-supervised TAME-GT

As described in Section 3.4, the TAME-GT variant replaces the pseudo-labeled TA validation set with the small set of ground-truth labeled traces. It naturally follows that TAME-GT serves as a natural performance upper bound for the blind TAME. In this section we quantify the gap between models with access to partial known labels and those without such prior label information and the results are presented in Table 3.

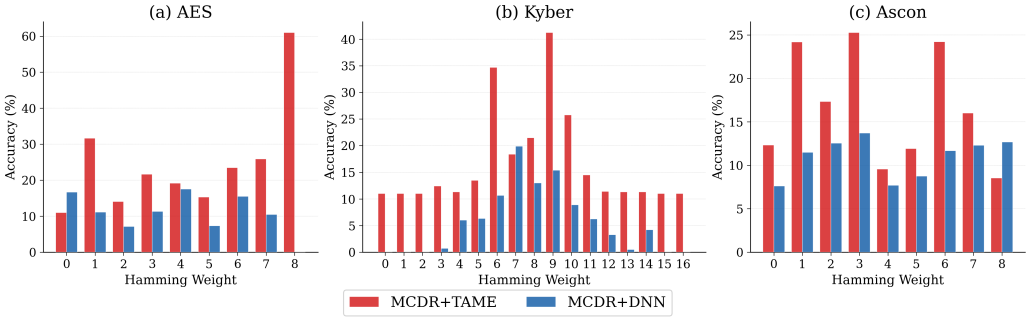


Figure 4: Label accuracy on the attack set under DNN and TAME framework on AES, Kyber, and ASCON datasets.

Table 3: Performance comparison with semi-supervised TAME-GT.

Datasets	AES			Kyber			ASCON		
	Without DNN	MLP	CNN	Without DNN	MLP	CNN	Without DNN	MLP	CNN
MCDR	GE=26	-	-	GE=195	-	-	GE=47	-	-
MCDR+DNN	-	GE=8	GE=7	-	GE=188	GE=171	-	GE=3	GE=3
MCDR+TAME	-	660	GE=3	-	5546	GE=1	-	7605	GE=1
MCDR+TAME-GT	-	346	580	-	1402	GE=1	-	786	GE=1

Figure 5 illustrates the performance gaps between existing frameworks including without DNN and DNN approaches, our proposed TAME framework, and the semi-supervised upper bound TAME-GT across three algorithm datasets. Our earlier analytical results have verified the superiority of MCDR analysis so we uniformly adopt the MCDR labeling method in this experiment. The figure shows that TAME-GT achieves the best performance and this outcome is expected as it has access to the true labels of a small number of samples. Notably even without access to correct labels, the TAME framework maintains a very small gap from the semi-supervised upper bound and successfully recovers the secret key in all cases.

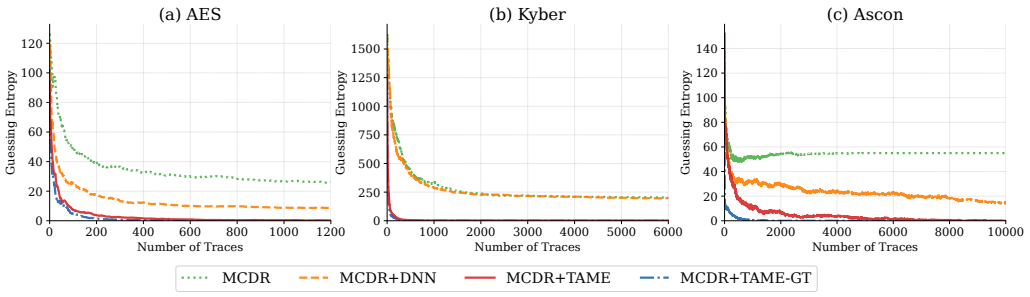


Figure 5: GE evolution comparing gaps between semi-supervised TAME-GT and blind frameworks under MCDR labeling method.

4.4 Results against countermeasures

4.4.1 Desynchronization countermeasure

Previous blind SCA approaches typically rely on a single PoI per variable. However, hiding countermeasures such as desynchronization disrupt trace alignment, making it difficult

for the attacker to identify a single informative PoI. To address this, Rezaeezade et al. introduced a DNN-based framework that considers multiple PoIs (50 per variable), and demonstrated, for the first time, the applicability of blind SCA under desynchronization.

In [RYJ+25], random shifts of up to 10 samples are applied to traces in either left or right direction based on an unprotected AES implementation. To further investigate how the degree of desynchronization affects attack performance, we conduct multiple groups of desynchronization experiments on the unprotected AES implementation, with the maximum number of random shift samples set to $\{4, 10, 16, 22, 28, 200\}$.

We then compare our TAME framework combined with MCDR labeling against three prior methods, as reported in Table 4. For neural network–based approaches, the reported results correspond to the best-performing model selected from 100 randomly sampled architectures in the search space[6]. When the validation set size $N_v = 0$, our framework reduces to the MCDR + DNN, so we evaluate $N_v \in \{0, 5, 10, \ldots, 200\}$.

The results show that our method achieves the best performance under all desynchronization levels. Specifically, when the desynchronization level is below 16, we successfully reduce GE to 0. At desynchronization levels of 22 and 28, the GE values remain relatively low, indicating that key recovery is highly promising by increasing the number of attack traces. When the desynchronization level reaches 200, the GE of our method increases much more slowly compared with other approaches, demonstrating its effectiveness in resisting desynchronization.

We additionally observe that at a desynchronization level of 10 with $N_v = 0$, key recovery requires only 150 traces. This phenomenon benefits from the fact that MCDR trains an encoder for variable-length sequences. By introducing small random shifts during training, the neural network is forced to learn waveform shapes and relative leakage patterns rather than fixed sample positions. This improves the model's robustness against desynchronization and reduces overfitting.

Table 4: Performance comparison on desynchronized AES.

Desynchronization level	4	10	16	22	28	200
Slicing [LDLL14]	GE=138	GE=99	GE=104	GE=128	GE=137	GE=134
VA [CR17]	GE=131	GE=107	GE=113	GE=130	GE=137	GE=127
MC+DNN [RYJ+25]	GE=13	GE=10	GE= 21	GE=3	GE=27	GE=123
MCDR+TAME (this work)	1129	150	1311	GE=1	GE=9	GE=37

4.4.2 Masking countermeasure

In this subsection, we conducted experiments on the public ASCAD dataset [BPS+20], which contains power traces collected from an AVR AES implementation protected by first-order Boolean masking. Using both the SOTA method and our proposed TAME approach, we explored the feasibility of blind SCA for evaluating masked implementations. Specifically, we used the ASCADv1 fixed-key profiling dataset and considered four sensitive intermediate values as attack targets: the third byte of the masked input share $m_1 = p \oplus k^* \oplus r_{in}$, the corresponding mask share $m_2 = r_{in}$, the third byte of the masked S-box output $y = \text{Sbox}[p \oplus k^*] \oplus r_{out}$, and the output mask share $m_3 = r_{out}$, where p denotes the plaintext byte, and r_{in} and r_{out} denote the input and output masking values, respectively. These variables satisfy the relation $y = \text{Sbox}[m_1 \oplus m_2] \oplus m_3$.

Because the analysis involves four target variables, the SOTA method would require clustering 6561 classes in a high-dimensional (200-dimensional) raw-PoI space. During GMM fitting, this entails estimating 6561 covariance matrices of dimension 200, resulting in severe memory and computational bottlenecks. Due to these limitations, a 6561-component

[6]We conduct experiments under the MLP framework.

GMM is practically infeasible, and our available computational resources are insufficient to complete the analysis.

In contrast, the TAME approach employs the DTCR labeling method to map the 200-dimensional PoIs into a fixed-dimensional representation space, followed by K-means clustering on these representations. Since K-means only requires centroid computations and does not involve covariance estimation, it completely bypasses the computational bottlenecks associated with GMM. As a result, TAME effectively addresses the dimensionality explosion problem and is able to complete the analysis within a practical time frame. But TAME gets a GE of 221 with 8000 training trances and 2000 attack traces. These results suggest that recovering keys from masked implementations without a profiling device and without knowledge of plaintexts/ciphertexts remains a highly challenging problem. We believe this is an important research direction that deserves further investigation in future work.

4.5 Results under different noise conditions

To further evaluate the robustness of the proposed framework under higher noise conditions, we inject six levels of additive Gaussian noise into the unprotected AES implementation dataset, with standard deviations of $[0.1, 0.15, 0.2, 0.25, 0.3, 0.35]$. We compare our TAME framework combined with MCDR labeling against SOTA method in Table 5. The reported results correspond to the best-performing model selected from 100 randomly sampled architectures in the search space under MLP and CNN networks.

The results demonstrate that our method consistently outperforms SOTA across all noise levels, indicating superior noise robustness compared to SOTA methods. Furthermore, when the noise variance is below 0.3, our method successfully recovers the key in all cases.

Table 5: Performance comparison when injecting different levels of Gaussian noise on the AES dataset.

Gaussian noise level	0.1	0.15	0.2	0.25	0.3	0.35
MC+DNN [RYJ+25]	GE=1	GE=8	GE= 1	GE=52	GE=14	GE=3
MCDR+TAME (this work)	728	829	955	1613	848	GE=1

4.6 Summary

Table 6 provides a summary comparing the limitations and capabilities of existing methods and the proposed approach. Single-point methods [LDLL14, CR17] analyze each variable independently from one PoI, provide no joint distribution estimate, and rely on strong linearity assumptions, which limits them to low-noise, univariate settings. [RYJ+25] overcomes the single-point restriction by performing joint clustering over multiple PoIs, and incorporates deep learning to refine pseudo-labels. However, it is constrained by two fundamental limitations: (1) the number of clusters $M = (B + 1)^{n_b}$ grows exponentially with the number of targeted variables, making GMM fitting unreliable for complex targets such as ASCON ($M = 729$) and Kyber ($M = 289$); and (2) all pseudo-labeled traces are treated with equal weight during training, ignoring the reliability gradient encoded in the clustering geometry. As a result, [RYJ+25] fails to recover the secret key on any of the three evaluated targets. The proposed framework addresses both limitations by replacing GMM with MCDR and introducing confidence-aware bilevel training (TAME), achieving successful key recovery across all targets, even under stronger desynchronization countermeasures and higher levels of noise injection.

Table 6: Limitations and capabilities of existing methods and the proposed approach.

Method	Multi-PoI per variable	Deep learning	Key recovery AES	Key recovery Kyber	Key recovery ASCON	Maximum level of desynchronization	Masking countermeasure	Maximum level of injecting noise
Slicing [LDLL14]	×	×	×	×	×	×	×	×
VA [CR17]	×	×	×	×	×	×	×	×
MC+DNN [RYJ+25]	✓	✓	✓	×	×	×	×	×
MCDR+TAME (this work)	✓	✓	✓	✓	✓	16	×	0.3

✓: supported / successful ×: not supported / failed N: maximum level for a successful attack

5 Conclusion

In this work, we identified two fundamental limitations of existing DL-BSCA method: the curse of dimensionality arising from multivariate clustering in the raw Hamming weight space, and the uniform treatment of traces during training that ignores the varying reliability of clustering-based labels. To address these limitations, we proposed TAME, a bilevel optimization framework that constructs a TA validation set from high-confidence cluster-interior traces and automatically learns per-sample weights guided by the validation loss on these anchors, thereby suppressing the adverse effect of mislabeled traces. Complementing TAME, we introduced MCDR labeling, which performs clustering in a compact learned feature space rather than the raw joint Hamming weight space, reducing noise sensitivity and alleviating dimensionality explosion. The two components are mutually reinforcing: better-separated clusters from MCDR yield higher-quality TA sets, which in turn strengthen TAME's meta-weight supervision. We further introduced TAME-GT, a variant that repurposes the traces used for PoI selection as a ground-truth validation set, establishing a tightly justified performance upper bound within the blind threat model. Experiments across AES, ASCON, and Kyber demonstrate that our framework recovers the secret key using at most 8,000 traces, while prior methods fail with guessing entropy consistently at least 3. Under desynchronization countermeasures, our method succeeds with as few as 150 traces where all existing approaches break down.

Despite these advances, several open challenges remain. First, existing blind SCA methods, including ours, assume that the attacker can precisely locate PoIs through correlation analysis. Relaxing this assumption and developing end-to-end blind SCA pipelines that jointly perform PoI identification and trace labeling would significantly broaden practical applicability. Second, masking is one of the most widely deployed countermeasures in cryptographic implementations, yet blind SCA against masked devices remains largely unexplored, as the interaction between unknown plaintext/ciphertext and share-splitting substantially complicates both the labeling and the distinguishing steps. Extending TAME and MCDR to the masked setting represents an important direction for future work. Finally, while our experiments focus on real measurements, systematic evaluation on a broader range of devices, leakage models, and post-quantum cryptographic primitives would further validate the generality of the proposed framework.

Acknowledgements

This work is supported in part by the National Natural Science Foundation of China (NSFC) under Grant Nos. 62502311 and U2336210, and in part by the Startup Fund for Young Faculty at SJTU (SFYF at SJTU).

References

[BCC+20] Sungyong Baik, Myungsub Choi, Janghoon Choi, Heewon Kim, and Kyoung Mu Lee. Meta-learning with adaptive hyperparameters. *Advances in neural information processing systems*, 33:20755–20765, 2020.

Table 7: Random search space for CNN

Hyperparameter	Search space
Convolutional layers	
Number of conv layers	$\{2, 3, 4\}$
Filters (layer 1)	$\{4, 8, 12, 16, 24\}$
Filters (layer $i > 1$)	$\text{filters}_{i-1} \times 2$
Conv kernel size	$\{4, 6, 8, 10, 12, 14, 16, 18\}$
Conv stride	$\{2, 4, 6, 8, 10\}$
Pooling type	{Average, Max}
Pooling kernel size	$\{4, 6, 8, 10\}$
Pooling stride	$\{4, 6, 8, 10\}$
Dense layers	
Number of dense layers	$\{2, 3\}$
Number of neurons per layer	$\{50, 100, 150, 200, 300, 400, 500\}$
Learning hyperparameters	
Weight initialization	{random_uniform, glorot_uniform, he_uniform}
Activation function	{relu, selu, elu, tanh}
Batch size	$\{128, 256, 512\}$
Learning rate	$\{10^{-3},\ 5\times10^{-4},\ 10^{-4},\ 5\times10^{-5},\ 10^{-5}\}$
Optimizer	Adam
Epochs	100

[BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In *Cryptographic Hardware and Embedded Systems-CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings 6*, pages 16–29. Springer, 2004.

[BLR12] Timo Bartkewitz and Kerstin Lemke-Rust. Efficient template attacks based on probabilistic multi-class support vector machines. In *International Conference on Smart Card Research and Advanced Applications*, pages 263–276. Springer, 2012.

[BN06] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.

[BPS+20] Ryad Benadjila, Emmanuel Prouff, Rémi Strullu, Eleonora Cagli, and Cécile Dumas. Deep learning for side-channel analysis and introduction to ascad database. *Journal of Cryptographic Engineering*, 10(2):163–188, 2020.

[BSTP21] David Basin, Ralf Sasse, and Jorge Toro-Pozo. The emv standard: Break, fix, verify. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1766–1781. IEEE, 2021.

[CK14] Marios O Choudary and Markus G Kuhn. Efficient stochastic methods: Profiled attacks beyond 8 bits. In *International Conference on Smart Card Research and Advanced Applications*, pages 85–103. Springer, 2014.

[CR17] Christophe Clavier and Léo Reynaud. Improved blind side-channel analysis by exploitation of joint distributions of leakages. In *International Conference on Cryptographic Hardware and Embedded Systems*, pages 24–44. Springer, 2017.

[CRR02] Suresh Chari, Josyula R Rao, and Pankaj Rohatgi. Template attacks. In *International workshop on cryptographic hardware and embedded systems*, pages 13–28. Springer, 2002.

[KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Advances in Cryptology—CRYPTO'99: 19th Annual International Cryptology*

Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19, pages 388–397. Springer, 1999.

[KL17] Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions. In *International conference on machine learning*, pages 1885–1894. PMLR, 2017.

[LB14] Hélène Le Bouder. *UN FORMALISME UNIFIANT LES ATTAQUES PHYSIQUES SUR CIRCUITS CRYTOGRAPHIQUES ET SON EXPLOITATION AFIN DE COMPARER ET RECHERCHER DE NOUVELLES ATTAQUES*. PhD thesis, Ecole Nationale Supérieure des Mines de Saint-Etienne, 2014.

[LDLL14] Yanis Linge, Cécile Dumas, and Sophie Lambert-Lacroix. Using the joint distributions of a cryptographic function in side channel analysis. In *International Workshop on Constructive Side-Channel Analysis and Secure Design*, pages 199–213. Springer, 2014.

[MP00] Geoffrey J McLachlan and David Peel. *Finite mixture models*. John Wiley & Sons, 2000.

[MPP16] Houssem Maghrebi, Thibault Portigliatti, and Emmanuel Prouff. Breaking cryptographic implementations using deep learning techniques. In *International Conference on Security, Privacy, and Applied Cryptography Engineering*, pages 3–26. Springer, 2016.

[MZLC19] Qianli Ma, Jiawei Zheng, Sen Li, and Gary W Cottrell. Learning representations for time series clustering. *Advances in neural information processing systems*, 32, 2019.

[PWP22] Guilherme Perin, Lichao Wu, and Stjepan Picek. Exploring feature selection scenarios for deep learning-based side-channel analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 828–861, 2022.

[RJBC23] Prasanna Ravi, Dirmanto Jap, Shivam Bhasin, and Anupam Chattopadhyay. Machine learning based blind side-channel attacks on pqc-based kems-a case study of kyber kem. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 01–07. IEEE, 2023.

[RYJ+25] Azade Rezaeezade, Trevor Yap, Dirmanto Jap, Shivam Bhasin, and Stjepan Picek. Breaking the blindfold: Deep learning-based blind side-channel analysis. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 5777–5796, 2025.

[RZYU18] Mengye Ren, Wenyuan Zeng, Bin Yang, and Raquel Urtasun. Learning to reweight examples for robust deep learning. In *International conference on machine learning*, pages 4334–4343. PMLR, 2018.

[SLGH21] Jaewoong Shin, Hae Beom Lee, Boqing Gong, and Sung Ju Hwang. Large-scale meta-learning with continual trajectory shifting. *arXiv preprint arXiv:2102.07215*, 2021.

[SLP05] Werner Schindler, Kerstin Lemke, and Christof Paar. A stochastic model for differential side channel cryptanalysis. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 30–46. Springer, 2005.

[SMY09] François-Xavier Standaert, Tal G Malkin, and Moti Yung. A unified framework for the analysis of side-channel key recovery attacks. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 443–461. Springer, 2009.

[SXY+19] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-weight-net: Learning an explicit mapping for sample weighting. *Advances in neural information processing systems*, 32, 2019.

[Tim19] Benjamin Timon. Non-profiled deep learning-based side-channel attacks with sensitivity analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 107–131, 2019.

[TMC+21] Meltem Sonmez Turan, Kerry McKay, Donghoon Chang, Cagdas Calik, Lawrence Bassham, Jinkeon Kang, and John Kelsey. Status report on the second round of the nist lightweight cryptography standardization process, 2021.

[VCS09] Nicolas Veyrat-Charvillon and François-Xavier Standaert. Mutual information analysis: how, when and why? In *International workshop on cryptographic hardware and embedded systems*, pages 429–443. Springer, 2009.

[WRLG18] Yuhuai Wu, Mengye Ren, Renjie Liao, and Roger Grosse. Understanding short-horizon bias in stochastic meta-optimization. *arXiv preprint arXiv:1803.02021*, 2018.

[Yu26] Jintong Yu. Blind leakage: Rethinking deep learning-based non-profiled side-channel analysis. *Cryptology ePrint Archive*, 2026.

[YWQ+25] Jintong Yu, Yuxuan Wang, Shipei Qu, Yubo Zhao, Yipeng Shi, Pei Cao, Xiangjun Lu, Chi Zhang, and Dawu Gu. End-to-end non-profiled side-channel analysis on long raw traces. In *European symposium on research in computer security*. Springer, 2025.